

Cây phủ tối thiểu và Union-Find (Minimum Spanning Trees and Disjoint Set Union)

Giảng viên: Trần Đình Tân

Môn: Thuật toán ứng dụng

Sau bài học, người học có thể:

- Phát biểu chính xác bài toán cây phủ tối thiểu trên đồ thị vô hướng có trọng số.
- Phân biệt MST với đường đi ngắn nhất, cây phủ bất kỳ và rừng phủ nhỏ nhất.
- Giải thích được tính chất lát cắt, tính chất chu trình và khái niệm cạnh an toàn.
- Mô phỏng, chứng minh đúng đắn và cài đặt được Kruskal với Union-Find.
- Mô phỏng, chứng minh đúng đắn và cài đặt được Prim với hàng đợi ưu tiên.
- Nhận biết điều kiện áp dụng, độ phức tạp, trường hợp biên và lỗi cài đặt thường gặp.

Nội dung

- 1 Bài toán MST
- 2 Tính chất tham lam
- 3 Kruskal và Union-Find
- 4 Prim
- 5 Ví dụ chạy tay
- 6 So sánh và lỗi thường gặp
- 7 Bài tập và tài liệu

Thiết lập

Xét đồ thị vô hướng có trọng số $G = (V, E)$, trong đó mỗi cạnh $e \in E$ có trọng số $w(e) \in \mathbb{R}$.

- Một **cây** là đồ thị liên thông và không chứa chu trình.
- Một **cây phủ** của G là cây con chứa toàn bộ đỉnh trong V .
- Nếu $|V| = n$, mọi cây phủ có đúng $n - 1$ cạnh.
- Nếu đồ thị không liên thông thì không tồn tại cây phủ; khi đó ta xét **rừng phủ nhỏ nhất**.

Định nghĩa cây phủ tối thiểu

Bài toán MST

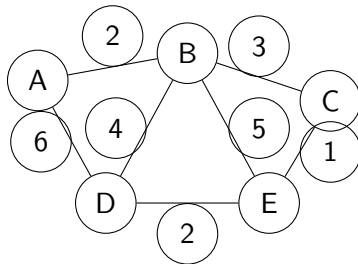
Cho đồ thị vô hướng, liên thông, có trọng số $G = (V, E)$. Tìm tập cạnh $T \subseteq E$ sao cho (V, T) là cây và

$$w(T) = \sum_{e \in T} w(e)$$

là nhỏ nhất.

- MST tối ưu theo **tổng trọng số toàn cục**.
- MST không yêu cầu trọng số không âm; trọng số âm vẫn được xử lý bình thường.
- MST có thể không duy nhất.

Ví dụ mở đầu



Câu hỏi thảo luận

Muốn kết nối tất cả các đỉnh với tổng chi phí nhỏ nhất, ta nên chọn những cạnh nào? Vì sao không chỉ chọn các cạnh nhỏ nhất một cách độc lập?

MST không phải là đường đi ngắn nhất

MST

- Chọn tập cạnh để nối tất cả đỉnh.
- Tối thiểu hóa tổng trọng số của cây.
- Không có đỉnh nguồn cố định.

Shortest paths

- Tìm đường đi ngắn từ một nguồn đến các đỉnh.
- Tối ưu từng khoảng cách từ nguồn.
- Có thể tạo cây đường đi ngắn nhất, nhưng không nhất thiết là MST.

Lưu ý

Một MST không đảm bảo đường đi giữa hai đỉnh bất kỳ trong cây là đường đi ngắn nhất trong đồ thị gốc.

Tính chất của nghiệm MST

- Một nghiệm hợp lệ phải nối tất cả đỉnh và không chứa chu trình.
- Nếu thêm bất kỳ cạnh ngoài cây vào một cây phủ, ta tạo đúng một chu trình.
- Nếu bỏ bất kỳ cạnh trong cây, cây bị tách thành hai thành phần.
- Nếu tất cả trọng số cạnh đôi một khác nhau thì MST là duy nhất.
- Ngược lại, nếu có cạnh bằng trọng số thì MST có thể duy nhất hoặc không duy nhất; cần xét cấu trúc đồ thị cụ thể.

Ứng dụng và giới hạn mô hình

- Thiết kế mạng kết nối chi phí thấp: cáp, đường truyền, đường ống, tuyến hạ tầng.
- Phân cụm dữ liệu: MST liên quan chặt với single-linkage clustering.
- Thiết kế mạng cảm biến, mạng giao thông phụ trợ, kết nối điểm dịch vụ.
- Tiên xử lý trong một số bài toán tối ưu tổ hợp và đồ thị.

Giới hạn

MST chỉ mô hình hóa yêu cầu “kết nối tất cả đỉnh với tổng chi phí nhỏ nhất”. Nếu có hướng cạnh, dung lượng, độ tin cậy, độ trễ, cân bằng tải hoặc nhiều mục tiêu, cần mô hình mở rộng khác.

Vì sao có thể dùng tham lam?

Câu hỏi trung tâm

Khi nào việc chọn một cạnh “nhẹ” ở thời điểm hiện tại là an toàn cho nghiệm tối ưu toàn cục?

- Không phải mọi lựa chọn cục bộ nhỏ nhất đều đúng trong mọi bài toán tối ưu.
- MST có cấu trúc đặc biệt: lát cắt và chu trình cho phép chứng minh lựa chọn tham lam.
- Hai thuật toán Kruskal và Prim khác nhau về cách chọn lát cắt, nhưng cùng dựa trên một nguyên lý: **chọn cạnh an toàn**.

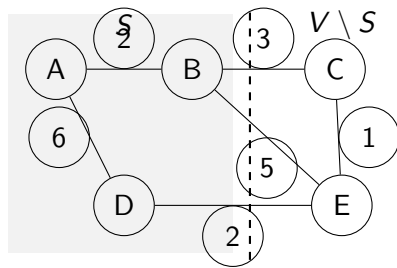
Lát cắt và cạnh băng qua lát cắt

Lát cắt

Một lát cắt là một phân hoạch $V = S \cup (V \setminus S)$, với $S \neq \emptyset$ và $S \neq V$.

- Cạnh (u, v) **băng qua lát cắt** nếu $u \in S$ và $v \in V \setminus S$, hoặc ngược lại.
- Cạnh nhẹ nhất qua lát cắt là cạnh có trọng số nhỏ nhất trong các cạnh băng qua lát cắt.
- Có thể có nhiều cạnh nhẹ nhất nếu trọng số bằng nhau.

Minh họa lát cắt



- Với $S = \{A, B, D\}$, các cạnh băng qua lát cắt gồm $BC(3)$, $BE(5)$, $DE(2)$.
- Cạnh nhẹ nhất qua lát cắt là $DE(2)$.

Định lý lát cắt

Gọi A là tập cạnh đã chọn và giả sử A là tập con của một MST nào đó. Nếu lát cắt $(S, V \setminus S)$ không bị cạnh nào trong A băng qua, thì mọi cạnh nhẹ nhất băng qua lát cắt đó là **cạnh an toàn** cho A .

- “An toàn” nghĩa là sau khi thêm cạnh đó vào A , vẫn tồn tại một MST chứa tập cạnh mới.
- Điều kiện “lát cắt không bị cạnh nào trong A băng qua” rất quan trọng.
- Không được phát biểu sai rằng “mọi cạnh nhẹ nhất của mọi lát cắt đều thuộc mọi MST”.

Ý tưởng chứng minh định lý lát cắt

- 1 Lấy một MST T chứa A .
- 2 Gọi e là cạnh nhẹ nhất băng qua lát cắt đang xét.
- 3 Nếu $e \in T$, kết luận hiển nhiên.
- 4 Nếu $e \notin T$, thêm e vào T sẽ tạo đúng một chu trình.
- 5 Chu trình đó phải chứa một cạnh f khác cũng băng qua lát cắt.
- 6 Vì e nhẹ nhất nên $w(e) \leq w(f)$; thay f bằng e không làm tăng tổng trọng số.

Kết luận

Tồn tại một MST chứa $A \cup \{e\}$, nên e là cạnh an toàn.

Định lý chu trình

Với một chu trình bất kỳ, nếu cạnh e là cạnh có trọng số lớn nhất một cách nghiêm ngặt trên chu trình đó, thì e không thuộc bất kỳ MST nào.

- Nếu cạnh nặng nhất không duy nhất, kết luận “không thuộc bất kỳ MST nào” có thể sai.
- Trực giác: trong một chu trình, bỏ đi một cạnh vẫn giữ được liên thông giữa các đỉnh trong chu trình.
- Nếu có một cạnh rõ ràng nặng hơn tất cả cạnh còn lại trên chu trình, giữ nó là không cần thiết cho tối ưu.

Hai nguyên lý thường dùng

Nguyên lý	Vai trò trong MST
Tính chất lát cắt	Cho biết cạnh nào có thể thêm vào mà vẫn an toàn.
Tính chất chu trình	Cho biết cạnh nào có thể loại bỏ vì chắc chắn không cần thiết trong nghiệm tối ưu.

Thông điệp

Kruskal thường được nhìn như quá trình thêm cạnh an toàn; đồng thời, mỗi cạnh bị bỏ qua cũng có thể giải thích bằng tính chất chu trình.

Chiến lược

Xét các cạnh theo thứ tự trọng số tăng dần. Thêm một cạnh nếu nó nối hai thành phần liên thông khác nhau; bỏ qua nếu nó tạo chu trình.

- Tập cạnh đang chọn luôn là một rừng.
- Mỗi cạnh được thêm sẽ giảm số thành phần liên thông đi 1.
- Thuật toán dừng khi đã chọn đủ $|V| - 1$ cạnh hoặc khi hết cạnh.
- Nếu đồ thị liên thông, kết quả là một MST; nếu không liên thông, kết quả là một MSF.

Kruskal: pseudocode

- ❶ Khởi tạo $T \leftarrow \emptyset$.
- ❷ Với mỗi đỉnh $v \in V$, gọi $\text{MAKE-SET}(v)$.
- ❸ Sắp xếp các cạnh E theo trọng số không giảm.
- ❹ Với mỗi cạnh (u, v) theo thứ tự đã sắp xếp:
 - ❶ Nếu $\text{FIND}(u) \neq \text{FIND}(v)$, thêm (u, v) vào T .
 - ❷ Gọi $\text{UNION}(u, v)$ để hợp nhất hai thành phần.
- ❺ Nếu $|T| = |V| - 1$, trả về T ; ngược lại, báo đồ thị không liên thông và trả về rừng phủ nhỏ nhất nếu bài toán cho phép.

Vì sao Kruskal đúng?

- Tại một bước, Kruskal chọn cạnh nhẹ nhất $e = (u, v)$ nối hai thành phần khác nhau của rừng hiện tại.
- Gọi S là tập đỉnh trong thành phần chứa u .
- Không có cạnh đã chọn nào băng qua lát cắt $(S, V \setminus S)$, vì các cạnh đã chọn chỉ nằm trong từng thành phần.
- Cạnh e là một cạnh nhẹ nhất băng qua một lát cắt phù hợp với tập cạnh đã chọn.
- Theo định lý lát cắt, e là cạnh an toàn.

Kết luận

Mỗi bước của Kruskal giữ bất biến: tập cạnh đã chọn là tập con của một MST nào đó.

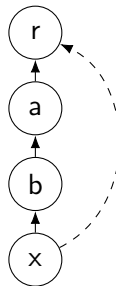
Mục đích

Duy trì phân hoạch các đỉnh thành các thành phần liên thông trong quá trình Kruskal.

- $\text{MAKE-SET}(v)$: tạo tập riêng chứa v .
- $\text{FIND}(v)$: trả về đại diện của tập chứa v .
- $\text{UNION}(u, v)$: hợp nhất hai tập chứa u và v .
- Hai kỹ thuật chuẩn: **path compression** và **union by rank/size**.

Trực giác về nén đường đi

- Mỗi tập được biểu diễn bằng một cây gốc.
- $\text{FIND}(x)$ đi từ x lên gốc để tìm đại diện.
- Nén đường đi làm cho các đỉnh trên đường tìm kiếm trở trực tiếp về gốc.
- Kết quả: các lần tìm kiếm sau nhanh hơn đáng kể.



Độ phức tạp của Union–Find

- Nếu cài đặt ngây thơ, cây đại diện có thể rất cao và FIND có thể chậm.
- Với path compression và union by rank/size, dãy p phép toán trên n phần tử chạy trong

$$\mathcal{O}(p \alpha(n)),$$

trong đó $\alpha(n)$ là hàm Ackermann nghịch đảo.

- Trong thực tế, $\alpha(n)$ tăng cực chậm và thường được xem là gần hằng số.
- Tuy nhiên, phát biểu chính xác vẫn là thời gian khấu hao, không phải mỗi phép toán riêng lẻ luôn hằng số tuyệt đối.

Cài đặt DSU trong C++: khởi tạo và Find

```
struct DSU {  
    vector<int> parent, sz;  
  
    DSU(int n) : parent(n), sz(n, 1) {  
        iota(parent.begin(), parent.end(), 0);  
    }  
  
    int find(int x) {  
        if (parent[x] == x) return x;  
        return parent[x] = find(parent[x]);  
    }  
}
```

- `parent[x]` lưu cha của `x` trong cây đại diện.
- Dòng gán trong `find` chính là nén đường đi.

Cài đặt DSU trong C++: Union by size

```
bool unite(int a, int b) {  
    a = find(a); b = find(b);  
    if (a == b) return false;  
    if (sz[a] < sz[b]) swap(a, b);  
    parent[b] = a;  
    sz[a] += sz[b];  
    return true;  
}  
};
```

- unite trả về false nếu hai đỉnh đã cùng thành phần.
- Trong Kruskal, giá trị trả về này quyết định có chọn cạnh hay không.

Cài đặt Kruskal trong C++: sắp xếp cạnh

```
struct Edge {
    int u, v;
    long long w;
};

pair<long long, vector<Edge>> kruskal(int n, vector<Edge> edges) {
    sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
        return a.w < b.w;
    });

    DSU dsu(n);
    long long total = 0;
    vector<Edge> mst;
```

Cài đặt Kruskal trong C++: chọn cạnh

```
for (const Edge& e : edges) {  
    if (dsu.unite(e.u, e.v)) {  
        mst.push_back(e);  
        total += e.w;  
        if ((int)mst.size() == n - 1) break;  
    }  
}  
return {total, mst};  
}
```

- Sau khi chạy, cần kiểm tra `mst.size() == n - 1` để xác nhận đồ thị liên thông.

Kiểm tra kết quả Kruskal

Sau khi chạy Kruskal, cần kiểm tra:

- Nếu $n = 0$: đầu vào không hợp lệ hoặc cần quy ước riêng.
- Nếu $n = 1$: MST rỗng, tổng trọng số bằng 0.
- Nếu $|T| = n - 1$: đồ thị liên thông và T là MST.
- Nếu $|T| < n - 1$: đồ thị không liên thông; T là rừng phủ nhỏ nhất của các thành phần liên thông.

Lỗi thường gặp

Chỉ trả về tổng trọng số mà không kiểm tra số cạnh có thể khiến chương trình gọi nhầm rừng là cây phủ.

Độ phức tạp của Kruskal

Ký hiệu $n = |V|$, $m = |E|$.

- Sắp xếp cạnh: $\mathcal{O}(m \log m)$.
- Các thao tác Union-Find: $\mathcal{O}((n + m)\alpha(n))$.
- Tổng thời gian: $\mathcal{O}(m \log m)$.
- Với đồ thị đơn, do $m \leq n(n - 1)/2$, có thể viết $\mathcal{O}(m \log n)$.
- Bộ nhớ phụ: $\mathcal{O}(n)$ cho DSU, ngoài danh sách cạnh đầu vào và kết quả.

Prim: ý tưởng thuật toán

Chiến lược

Bắt đầu từ một đỉnh bất kỳ. Mỗi bước thêm cạnh nhẹ nhất nối cây hiện tại với một đỉnh chưa thuộc cây.

- Prim duy trì đúng một cây đang mở rộng trong đồ thị liên thông.
- Lát cắt tại mỗi bước là “đỉnh đã thuộc cây” và “đỉnh chưa thuộc cây”.
- Thuật toán dừng khi đã đưa tất cả đỉnh vào cây.

Prim: pseudocode với hàng đợi ưu tiên

- ❶ Chọn đỉnh bắt đầu s .
- ❷ Khởi tạo $key[s] \leftarrow 0$; với mọi $v \neq s$, đặt $key[v] \leftarrow +\infty$ và $parent[v] \leftarrow \text{NIL}$.
- ❸ Đưa các đỉnh vào hàng đợi ưu tiên theo giá trị key .
- ❹ Trong khi hàng đợi chưa rỗng:
 - ❶ Lấy u có $key[u]$ nhỏ nhất ra khỏi hàng đợi.
 - ❷ Với mỗi cạnh (u, v) kề u , nếu v chưa được chọn và $w(u, v) < key[v]$, cập nhật $key[v]$ và $parent[v]$.
- ❺ Trả về các cạnh $(parent[v], v)$ với $v \neq s$.

Vì sao Prim đúng?

- Giả sử S là tập đỉnh đã nằm trong cây hiện tại.
- Prim chọn cạnh nhẹ nhất băng qua lát cắt $(S, V \setminus S)$.
- Tập cạnh đã chọn nằm hoàn toàn trong S , nên không có cạnh đã chọn nào băng qua lát cắt đó.
- Theo định lý lát cắt, cạnh được chọn là cạnh an toàn.

Kết luận

Lặp lại đến khi tất cả đỉnh được chọn, Prim thu được một MST.

Cài đặt Prim với priority queue: khởi tạo

```
long long prim(int n, vector<vector<pair<int,int>>>& adj) {
    const long long INF = (1LL << 60);
    vector<long long> key(n, INF);
    vector<int> used(n, 0);
    priority_queue<pair<long long,int>,
        vector<pair<long long,int>>, greater<pair<long long,int>>> pq;

    key[0] = 0;
    pq.push({0, 0});
    long long total = 0;
    int cnt = 0;
```

- $key[v]$ là trọng số cạnh nhẹ nhất đang biết để nối v vào cây.
- $used[v]$ đánh dấu đỉnh đã được đưa vào MST.

Cài đặt Prim với priority queue: vòng lặp

```
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (used[u]) continue;
    used[u] = 1; total += d; cnt++;
    for (auto [v, w] : adj[u]) {
        if (!used[v] && w < key[v]) {
            key[v] = w;
            pq.push({key[v], v});
        }
    }
}
if (cnt < n) throw runtime_error("Graph is disconnected");
return total;
}
```

Một chi tiết cài đặt Prim

- Trong C++ `priority_queue` chuẩn không hỗ trợ thao tác decrease-key trực tiếp.
- Cách phổ biến: mỗi lần cập nhật $key[v]$, đẩy một bản ghi mới vào heap.
- Khi lấy ra, nếu đỉnh đã được chọn thì bỏ qua bản ghi cũ.
- Cách này vẫn đạt $\mathcal{O}(m \log m)$, thường được viết $\mathcal{O}(m \log n)$ trong đồ thị đơn vì số bản ghi heap tỷ lệ với số cạnh.

Độ phức tạp của Prim

- Ma trận kề, chọn min tuyến tính: $\mathcal{O}(n^2)$; phù hợp với đồ thị dày.
- Danh sách kề và binary heap: $\mathcal{O}(m \log n)$ nếu có decrease-key; với kỹ thuật đẩy bản ghi mới, có thể viết $\mathcal{O}(m \log m)$.
- Với Fibonacci heap: $\mathcal{O}(m + n \log n)$ về lý thuyết, nhưng cài đặt phức tạp hơn.
- Nếu đồ thị không liên thông, chạy Prim từ từng thành phần sẽ cho rừng phủ nhỏ nhất.

Ví dụ 1: bảng cạnh

Xét $V = \{A, B, C, D, E\}$ và các cạnh có trọng số:

Cạnh	AB	AC	BC	BD	CD	CE	DE
Trọng số	1	3	2	4	5	6	7

Yêu cầu

Mô phỏng Kruskal và Prim. So sánh thứ tự chọn cạnh và tổng trọng số.

Mô phỏng Kruskal trên ví dụ 1

Các cạnh theo thứ tự tăng dần: $AB(1)$, $BC(2)$, $AC(3)$, $BD(4)$, $CD(5)$, $CE(6)$, $DE(7)$.

Bước	Cạnh xét	Quyết định
1	$AB(1)$	Chọn: nối A và B
2	$BC(2)$	Chọn: nối thành phần $\{A, B\}$ với C
3	$AC(3)$	Bỏ qua: tạo chu trình $A - B - C - A$
4	$BD(4)$	Chọn: thêm D
5	$CD(5)$	Bỏ qua: C và D đã cùng thành phần
6	$CE(6)$	Chọn: thêm E và đủ $n - 1 = 4$ cạnh

MST thu được: $\{AB, BC, BD, CE\}$, tổng trọng số $1 + 2 + 4 + 6 = 13$.

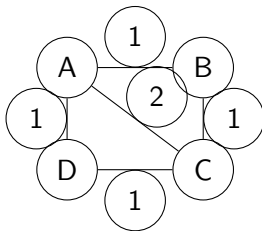
Mô phỏng Prim trên ví dụ 1

Bắt đầu từ đỉnh A.

Bước	Cạnh được chọn	Giải thích
1	$AB(1)$	Nhẹ nhất từ $\{A\}$ ra ngoài
2	$BC(2)$	Nhẹ nhất từ $\{A, B\}$ ra ngoài
3	$BD(4)$	Nhẹ nhất từ $\{A, B, C\}$ ra ngoài
4	$CE(6)$	Nhẹ nhất từ $\{A, B, C, D\}$ ra ngoài

- Kết quả trùng với Kruskal trong ví dụ này.
- Trong các đồ thị khác, thứ tự chọn cạnh có thể khác nhưng tổng trọng số tối ưu vẫn giống nhau.

Ví dụ 2: nhiều MST



- Bốn cạnh ngoài của hình vuông đều có trọng số 1.
- Bỏ một cạnh bất kỳ trong chu trình bốn cạnh ngoài, ta được một cây phủ có tổng trọng số 3.
- Vì có nhiều cạnh bằng trọng số, MST không duy nhất.

Ví dụ 3: trọng số âm

Cạnh	AB	AC	BC	BD	CD
Trọng số	-4	2	1	3	-1

- Kruskal vẫn sắp xếp theo trọng số tăng dần: $AB(-4)$, $CD(-1)$, $BC(1)$, $AC(2)$, $BD(3)$.
- Trọng số âm không phá vỡ định lý lát cắt hay định lý chu trình.
- MST có thể có tổng trọng số âm; điều này hoàn toàn hợp lệ.

So sánh Kruskal và Prim

Tiêu chí	Kruskal	Prim
Góc nhìn	Xét cạnh toàn cục	Mở rộng từ một cây
Cấu trúc dữ liệu chính	Union-Find	Hàng đợi ưu tiên / ma trận kề
Đầu vào thuận lợi	Danh sách cạnh	Danh sách kề hoặc ma trận kề
Phù hợp	Đồ thị thưa, cạnh dễ sắp xếp	Đồ thị dày hoặc cần phát triển từ một đỉnh
Cơ sở đúng đắn	Lát cắt giữa các thành phần	Lát cắt giữa cây và phần còn lại

Khi nào nên dùng thuật toán nào?

- Nếu dữ liệu đầu vào là danh sách cạnh và đồ thị thưa: Kruskal thường đơn giản và hiệu quả.
- Nếu đồ thị dày và đã có ma trận kề: Prim với ma trận kề $\mathcal{O}(n^2)$ có thể phù hợp.
- Nếu cần xử lý từng thành phần hoặc dễ quản lý cạnh toàn cục: Kruskal thuận tiện.
- Nếu cần xây dựng cây từ một điểm bắt đầu cụ thể để minh họa quá trình mở rộng: Prim trực quan hơn.

Sai lầm khái niệm thường gặp

- Áp dụng MST trực tiếp cho đồ thị có hướng; đồ thị có hướng cần bài toán khác, ví dụ minimum arborescence.
- Nhầm MST với cây đường đi ngắn nhất.
- Nghĩ rằng MST yêu cầu trọng số không âm.
- Nghĩ rằng nếu có hai cạnh bằng trọng số thì MST chắc chắn không duy nhất.
- Quên rằng đồ thị không liên thông không có cây phủ.

Sai lầm cài đặt thường gặp

- Trong Kruskal, quên gọi UNION sau khi chọn cạnh.
- Trong DSU, viết FIND nhưng không nén đường đi, dẫn tới chạy chậm trên dữ liệu lớn.
- Trong Prim, cộng trọng số nhiều lần do không bỏ qua bản ghi heap cũ.
- Không kiểm tra đồ thị liên thông sau khi chạy thuật toán.
- Dùng kiểu `int` cho tổng trọng số khi dữ liệu có thể gây tràn số.

Kiểm thử trường hợp biên

- Đồ thị chỉ có một đỉnh: MST rỗng, tổng trọng số bằng 0.
- Đồ thị không liên thông: không tồn tại cây phủ; thuật toán có thể trả về rừng phủ nhỏ nhất.
- Nhiều cạnh cùng trọng số: có thể có nhiều MST hợp lệ.
- Trọng số âm: thuật toán vẫn đúng nếu đồ thị vô hướng và mục tiêu là tổng trọng số nhỏ nhất.
- Đa cạnh giữa cùng một cặp đỉnh: xử lý như các cạnh riêng biệt; thuật toán sẽ tự chọn cạnh phù hợp.

Bảng kiểm audit lời giải MST

- ❶ Đồ thị có vô hướng không?
- ❷ Đồ thị có liên thông không? Nếu không, yêu cầu là MST hay MSF?
- ❸ Tập cạnh trả về có đúng $n - 1$ cạnh không?
- ❹ Tập cạnh trả về có tạo chu trình không?
- ❺ Tổng trọng số được tính bằng kiểu dữ liệu đủ lớn không?
- ❻ Thuật toán có xử lý đúng trọng số âm và cạnh bằng trọng số không?

Một số biến thể liên quan

- **Minimum spanning forest:** áp dụng khi đồ thị không liên thông.
- **Maximum spanning tree:** đổi mục tiêu thành tổng trọng số lớn nhất; có thể sắp xếp giảm dần trong Kruskal.
- **Minimum arborescence:** biến thể trên đồ thị có hướng, không phải MST thông thường.
- **Constrained MST:** thêm ràng buộc bậc, màu, độ tin cậy hoặc ngân sách phụ; thường khó hơn đáng kể.

Hoạt động trên lớp

- ① Chia lớp thành hai nhóm: một nhóm chạy Kruskal, một nhóm chạy Prim trên cùng đồ thị.
- ② Mỗi nhóm ghi lại thứ tự chọn cạnh, cạnh bị bỏ qua và lý do.
- ③ So sánh kết quả: tập cạnh có thể khác không? Tổng trọng số có khác không?
- ④ Yêu cầu mỗi nhóm chỉ ra lát cắt tương ứng với một bước chọn cạnh.

Bài tập tự luyện

- 1 Với một đồ thị 6–8 đỉnh, mô phỏng Kruskal từng bước và ghi trạng thái DSU.
- 2 Mô phỏng Prim từ hai đỉnh bắt đầu khác nhau. So sánh thứ tự chọn cạnh và tổng trọng số.
- 3 Chứng minh ngắn gọn vì sao cạnh bị Kruskal bỏ qua tạo chu trình nên không cần thêm vào rừng hiện tại.
- 4 Cài đặt Kruskal và kiểm thử trên: đồ thị một đỉnh, đồ thị không liên thông, đồ thị có trọng số âm, đồ thị có nhiều MST.
- 5 So sánh thời gian chạy thực nghiệm của Kruskal và Prim trên đồ thị thưa và đồ thị dày.

Câu hỏi kiểm tra nhanh

- 1 Nếu tất cả trọng số cạnh khác nhau, MST có duy nhất không?
- 2 Nếu đồ thị có trọng số âm, Kruskal có còn đúng không?
- 3 Một cạnh nhẹ nhất qua một lát cắt có luôn thuộc mọi MST không?
- 4 Vì sao Prim có thể bắt đầu từ bất kỳ đỉnh nào trong đồ thị liên thông?
- 5 Nếu Kruskal trả về ít hơn $n - 1$ cạnh, điều đó nói gì về đồ thị?

Đáp án ngắn cho kiểm tra nhanh

- ① Có, nếu tất cả trọng số cạnh đôi một khác nhau thì MST duy nhất.
- ② Có, trọng số âm không làm sai thuật toán MST.
- ③ Không nhất thiết thuộc mọi MST; phát biểu đúng là cạnh nhẹ nhất qua lát cắt phù hợp là an toàn.
- ④ Vì định lý lát cắt áp dụng cho mọi lát cắt tạo bởi cây hiện tại và phần còn lại.
- ⑤ Đồ thị không liên thông, nên không tồn tại cây phủ.

- MST là bài toán chọn cây phủ có tổng trọng số nhỏ nhất trên đồ thị vô hướng liên thông.
- Định lý lát cắt là nền tảng cho cạnh an toàn và chiến lược tham lam.
- Định lý chu trình giúp giải thích vì sao một số cạnh có thể bị loại bỏ.
- Kruskal dùng sắp xếp cạnh và DSU để tránh chu trình.
- Prim dùng lát cắt giữa cây hiện tại và phần còn lại, thường cài đặt bằng hàng đợi ưu tiên.
- Cần kiểm tra liên thông, trường hợp biên, kiểu dữ liệu và giả thiết đồ thị.

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, chương Minimum Spanning Trees.
- J. Kleinberg, É. Tardos, *Algorithm Design*, phần Greedy Algorithms và MST.
- R. Sedgewick, K. Wayne, *Algorithms*, phần Undirected Graphs và Minimum Spanning Trees.
- R. E. Tarjan, các kết quả kinh điển về disjoint-set union và phân tích khẩu hao.